

PROGRAMACIÓN Y ESTRUCTURAS DE DATOS

TP 2

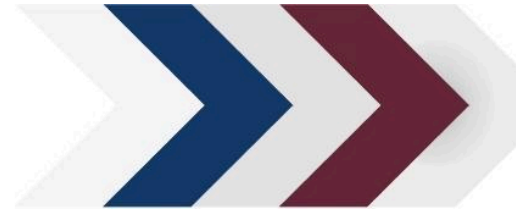
GUÍA DE REVISIÓN CONCEPTUAL

INTRODUCCIÓN A LA PROGRAMACIÓN VISUAL #2

UNIDAD 1

INTRODUCCIÓN A LA PROGRAMACIÓN VISUAL

Autor de contenidos:
Nicolás Battaglia



OBJETIVOS

Formalizar conceptos teóricos sobre la programación visual

ENUNCIADO

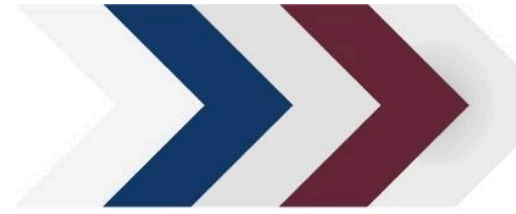
RESPONDA LAS SIGUIENTES PREGUNTAS

1. ¿Qué característica posee la programación visual
2. ¿Qué es un control de usuario?
3. ¿Cuáles son los tres elementos constitutivos que un control de usuario debe poseer?
4. ¿Para qué se utiliza la ventana denominada Solution Explorer?
5. ¿Para qué se utiliza la ventana denominada Properties?
6. ¿Para qué se utiliza la ventana denominada Toolbox?
7. ¿Qué es y que puede contener una Solución en el entorno de trabajo visto en clase?
8. ¿Qué es y que puede contener un Proyecto en el entorno de trabajo visto en clase?
9. ¿Qué es un formulario?
10. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control Button.
11. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control CheckBox.
12. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control Combobox.
13. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control DateTimePicker.
14. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control Timer.
15. Seleccione 10 controles adicionales y de cada uno de ellos describa 5 métodos, 5 propiedades y 5 eventos.

Parte 2

16. ¿Cómo se declara un procedimiento? Ejemplo.
17. ¿Cómo se declara una función? Ejemplo
18. ¿Qué diferencia existe entre una función y un procedimiento?



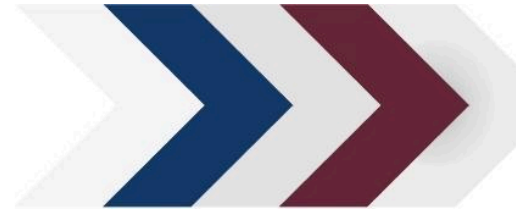


19. ¿Qué significa que el parámetro de un procedimiento se pasa por valor?
20. ¿Qué significa que el parámetro de una función se pasa por referencia?

Parte 3

21. Enumere las estructuras de decisión. Ejemplifique cada una de ellas y explique en que se diferencian.
22. Enumere las estructuras de repetición. Ejemplifique cada una de ellas y explique en que se diferencian.
23. ¿Cómo se crea una estructura en el entorno visto en clase?
24. ¿Qué es una propiedad?
25. ¿Qué es un método?
26. ¿Qué es un evento?
27. ¿Cuál es el uso ideal de una variable de tipo Boolean?
28. ¿Para que se usa el método Parse cuando se lo utiliza por ejemplo como Decimal.Parse?
29. ¿Cómo se declara un vector de 10 posiciones de tipo Decimal?
30. ¿Cuándo hablamos de boxing y unboxing a que nos referimos?





Respuestas:

1) La programación visual se caracteriza porque permite diseñar aplicaciones a través de una interfaz gráfica, arrastrando y soltando controles en un formulario, en lugar de escribir todo el código manualmente. Eso acelera el desarrollo y facilita la comprensión de la estructura de la app.

2) Un control de usuario es un componente visual que uno mismo diseña, combinando controles ya existentes, para reutilizarlo en distintos formularios o proyectos.

3) Una interfaz gráfica (lo que ve el usuario).

Propiedades (para configurarlo).

Eventos y métodos (para que pueda interactuar con el resto del programa).

4) Sirve para ver y administrar la estructura de la solución: proyectos, formularios, clases, recursos, referencias, etc.

5) Ahí se configuran las propiedades de un control o formulario seleccionado nombre, tamaño, color, fuente, etc.

6) La ventana Toolbox se utiliza para acceder a los controles y componentes disponibles en el entorno de desarrollo. Desde allí se pueden arrastrar y colocar elementos como botones, cuadros de texto, etiquetas, listas y otros controles sobre los formularios, facilitando el diseño de la interfaz gráfica de la aplicación.

7) Una Solución es el contenedor principal dentro de Visual Studio. Agrupa uno o varios proyectos que forman parte de un mismo sistema o aplicación. Puede contener:

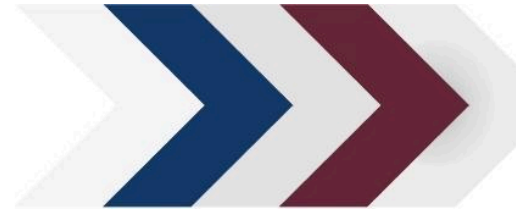
Proyectos de distintos tipos (Windows Forms, consola, librerías, etc.).

Archivos de configuración generales.

Recursos compartidos (imágenes, íconos, cadenas).

Referencias comunes entre proyectos.





8) Un Proyecto es la unidad básica dentro de una solución. Representa una aplicación o componente específico. Puede contener:

Archivos de código fuente en C#.

Formularios y sus archivos asociados.

Recursos propios (imágenes, sonidos, íconos).

Referencias a librerías externas.

Archivos de configuración particulares del proyecto.

9) Un Formulario es una ventana que sirve como interfaz gráfica en una aplicación Windows Forms. Es el espacio donde se colocan los controles (botones, cuadros de texto, etiquetas, etc.) que permiten la interacción entre el usuario y el programa. Cada formulario se implementa como una clase que hereda de `System.Windows.Forms.Form`.

10) Control Button

Propiedades (10): `Text`, `Enabled`, `Visible`, `BackColor`, `ForeColor`, `Font`, `Size`, `Location`, `Name`, `TabIndex` Métodos (10): `PerformClick()`, `Refresh()`, `Focus()`, `Hide()`, `Show()`, `Select()`,

11) Propiedades

1 `BackColor`

El color de fondo del componente.

2 `CausesValidation`

Indica si este componente genera eventos de validación.

3 `CheckAlign`

Determina la ubicación de la casilla dentro del control.

4 `FlatAppearance`

Para los botones cuyo `FlatStyle` es `FlatStyle.Flat`, determina la apariencia del borde y los colores utilizados para indicar el estado de activación el estado del mouse.

5 `Dock`

Define los bordes de control enlazados al contenedor.

6 `Enabled`

Indica si el control está habilitado.

7 `GenerateMember`

Indica si se generará una variable miembro para este componente.

8 `Image`



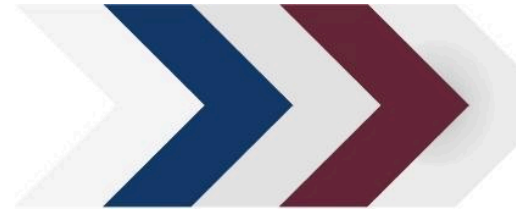


Imagen que se mostrará en el control

9 Locked

La propiedad si se puede mover o cambiar el tamaño del control.

10 ThreeState

Indica si CheckBox permitirá tres estados de activación en lugares de dos.

Metodos

Mycheckbox.Height = 50;

Mycheckbox.Width = 100;

Mycheckbox.Location = new Point(229, 136);

Mycheckbox.Text = "Hello World";

Mycheckbox.Font = new Font("Bradley Hand ITC", 12);

Focus(), Show(), Hide(), Refresh(), PerformClick(), Dispose(),

Eventos

CheckedChanged

Este evento ocurre cuando el valor de la propiedad Checked cambia

CheckStateChanged

Este evento ocurre cuando el valor de la propiedad CheckState cambia

Click

Este evento ocurre cuando se pulsa el control

DoubleClick

Este evento ocurre cuando el usuario hace doble click sobre el control

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

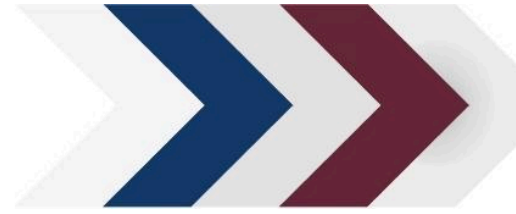
Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.



12) Propiedades

1 AutoCompleteSource

El origen de cadenas completas utilizadas para la finalización automática.

2 AutoCompleteMode

Indica el comportamiento de finalización del texto del cuadro combinado.

3 DrawMode

Indica si el código o el sistema operativo controlara el dibujo de elementos de la lista.

4 DropDownHeight

Alto, en píxeles, del cuadro desplegable en un cuadro combinado.

5 DropDownStyle

Comprueba la apariencia y funcionalidad del cuadro combinado.

6 DropDownWidth

Ancho, en píxeles, del cuadrado desplegable en un cuadro combinado.

7 MaxLength

Especifica el número máximo de caracteres que puede introducirse en un cuadro combinado.

8 Modifiers

Indica el nivel de visibilidad del objeto.

9 Stored

Especifica si están ordenados los elementos en la lista del cuadro combinado.

10 UseWaitCursor

Cuando esta propiedad es true, la propiedad cursos del control y sus controles secundarios se establece en waitcursor.

Metodos

```
ComboBox MyComboBox = new ComboBox();
```

```
mybox.Location = new Point(327, 77);
```

```
mybox.Size = new Size(216, 26);
```

```
mybox.Items.Add("C#");
```

```
mybox.Items.Add("Java");
```

```
mybox.Items.Add("Scala");
```

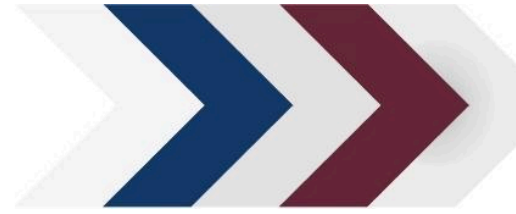
```
mybox.Items.Add("C");
```

```
mybox.Items.Add("C++");
```

```
Enabled;
```

```
Visible;
```





Eventos

Click

Este evento ocurre cuando se pulsa el control ComboBox.

DragDrop

Este evento ocurre cuando se completa una operación de arrastrar y soltar.

DropDown

Esto ocurre incluso cuando se muestra la parte desplegable de un ComboBox.

DropDownClosed

Este evento ocurre cuando la parte desplegable del ComboBox ya no es visible.

DropDownStyleChanged

Esto ocurre cuando la propiedad DropDownStyle ha cambiado.

Leave

Esto ocurre cuando el enfoque de entrada sale del control ComboBox.

MouseClicked

Esto ocurre cuando el ratón pulsa el control ComboBox.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control ComboBox.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control ComboBox y se pulsa un botón del ratón.

MouseEnter

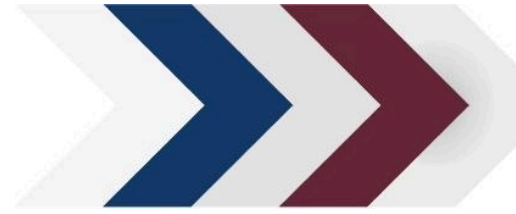
Este evento ocurre cuando el puntero del ratón entra en el control ComboBox.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control ComboBox.

SelectedIndexChanged

Este evento ocurre cuando la propiedad SelectedIndex ha cambiado.



13) DateTimePicker.

Propiedades

1 CalendarFont

Fuente utilizada para mostrar el calendario.

2 CalendarMonthBackground

Color de fondo que se muestra dentro del mes.

3 CalendarTitleBackColor

Color de fondo que se muestra en el titulo del calendario.

4 CalendarTitleForeColor

Color utilizado para mostrar texto en el titulo del calendario.

5 DropDownAling

Controla la alineación del calendario desplegable con el selector de fecha DateTimePicker. Puede tener los valores LEFT o RIGHT.

6 GenerarMember

Indica si se genera una variable miembro para este componente.

7 MaxDate

Fecha máxima que se puede seleccionar.

8 MinDate

Fecha mínima que se puede seleccionar.

9 Value

Valor de fecha y hora actual para ese control.

10 CausesValidation

Cuando este se encuentra en "True" indica que el componente puede validar eventos

Metodos

```
DateTimePicker d = new DateTimePicker();
```

```
d.Location = new Point(360, 162);
```

```
d.Size = new Size(292, 26);
```

```
d.MaxDate = new DateTime(2500, 12, 20);
```

```
d.MinDate = new DateTime(1753, 1, 1);
```

```
d.Format = DateTimePickerFormat.Long;
```

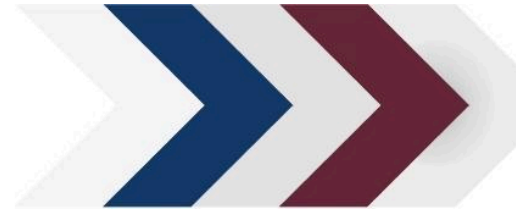
```
d.Name = "MyPicker";
```

```
d.Font = new Font("Comic Sans MS", 12);
```

```
d.Visible = true;
```

```
d.Value = DateTime.Today;
```





Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.

14) Propiedades

1 Enabled

Habilita la generacion de eventos "Elapsed"

2 Interval

Establece el intervalo entre Elapsed generados

3 Tag

Asocia una etiqueta elegida por el usuario

4 GenerateMember

Indica si una variable miembro va a ser generada para este componente

5 Modifiers

Indica el nivel de visibilidad del objeto

6 Name

Permite dar un nombre personalizado al objeto

Metodos

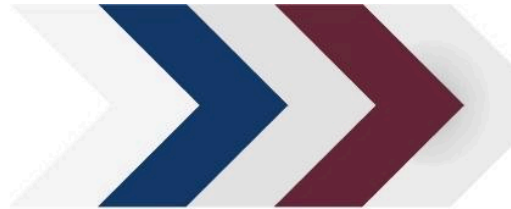
```
System.Timers.Timer aTimer = new System.Timers.Timer();
```

```
aTimer.Elapsed += new ElapsedEventHandler(OnTimedEvent);
```

```
aTimer.Interval = 5000; // ~ 5 seconds
```

```
aTimer.Enabled = true;
```





Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDown

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

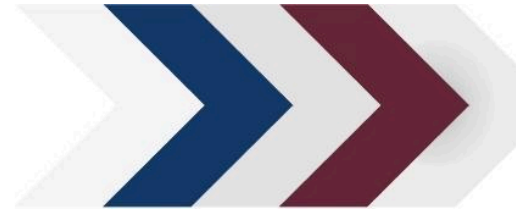
Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.



15) CheckedListBox

Propiedades

1 CheckOnClick

Indica si la casilla de cambiar su estado al hacer clic por primera vez en un elemento.

2 ColumnWidth

Indica el ancho que debe tener las columnas de un control ListBox de varias columnas.

3 FormatString

Caracteres de especificador de formato que indica como se muestra un valor.

4 FormattingEnabled

Si esta propiedad es true, el valor de FormatString se utiliza para convertir el valor de DisplayMember en un valor que se pueda mostrar.

5 GenerateMember

Indica si se genera una variable miembro para este componente.

Metodos

AddItemsCore(Object[])

BeginInvoke(Action)

BeginInvoke(Delegate, Object[])

BeginInvoke(Delegate)

BeginUpdate()

BringToFront()

ClearSelected()

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClick

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

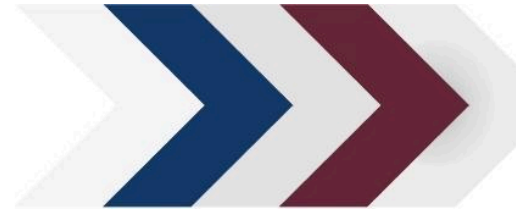
MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.





Button

Propiedades

1 Name

Indica el nombre utilizado en el código para identificar el objeto.

2 AccessibleDescription

Descripción que proporciona a clientes con discapacidades para facilitar el acceso.

3 AccessibleName

Nombre que se proporciona a clientes con discapacidades para facilitar al acceso.

4 AccessibleRole

Función que se proporciona a clientes con discapacidades para facilitar el acceso.

5 AllowDrop

Indica si el control puede aceptar datos que el usuario arrastre.

Métodos

AddChild(Object)

AddHandler(RoutedEvent, Delegate, Boolean)

AddHandler(RoutedEvent, Delegate)

AddLogicalChild(Object)

AddText(String)

AddToEventRoute(EventRoute, RoutedEventArgs)

AddVisualChild(Visual)

ApplyAnimationClock(DependencyProperty, AnimationClock, HandoffBehavior)

ApplyAnimationClock(DependencyProperty, AnimationClock)

ApplyTemplate()

Arrange(Rect)

ArrangeCore(Rect)

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDown

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

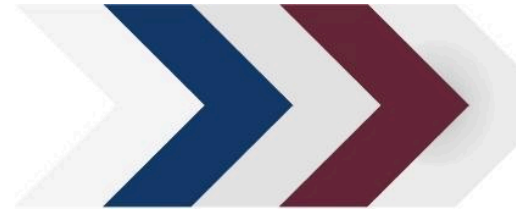
MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.





Label

Propiedades

1 AutoEllipsis

Permite el control automático del texto que se extiende mas alla del ancho del control de la etiqueta.

2 Padding

Especifica el espacio interior de un control.

3 RightToLeft

Indica si el componente debe dibujar de derecha a izquierda para idiomas RLT.

4 Tag

Datos definidos por el usuario asociados con el objeto.

5 UseMnemonic

Si es true, el primer carácter precedido por un símbolo de “y” comercial (&) se utiliza como la tecla de acceso de la etiqueta.

Metodos

```
Label mylab = new Label();  
mylab.Text = "GeeksforGeeks";  
mylab.Location = new Point(222, 90);  
mylab.AutoSize = true;  
mylab.Font = new Font("Calibri", 18);  
mylab.ForeColor = Color.Green;  
mylab.Padding = new Padding(6);
```

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

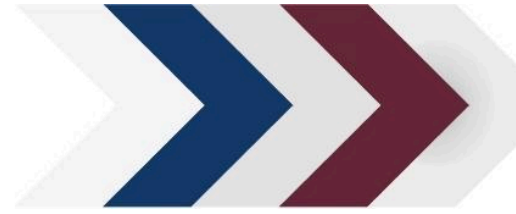
MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.





LinkLabel

Propiedades

1 activeLinkcolor

Determina el color del hipervínculo cuando el usuario hace clic en el vínculo.

2 DisableLinkColor

Determina el color del hipervínculo cuando esta deshabilitado.

3 LinkArea

Parte del texto de la etiqueta que se procesa como un hipervínculo.

4 LinkBehavior

Determina el comportamiento del subrayado de un hipervínculo.

5 LinkColor

Determina el color hipervínculo en el estado predeterminado.

Metodos

```
LinkLabel mylab = new Label();
```

```
mylab.Text = "GeeksforGeeks";
```

```
mylab.Location = new Point(222, 90);
```

```
mylab.AutoSize = true;
```

```
mylab.Font = new Font("Calibri", 18);
```

```
mylab.ForeColor = Color.Green;
```

```
mylab.Padding = new Padding(6);
```

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

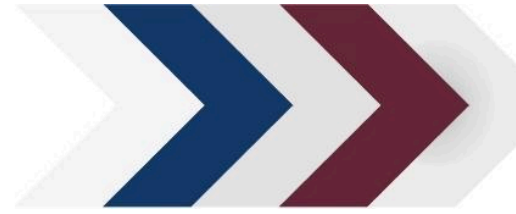
Este evento ocurre cuando el puntero del ratón descansa sobre el control.

ListBox

Propiedades

1 IntegralHeight

Indica si la lista puede contener solo elementos completos.



2 Items

Elementos en el cuadro de lista

3 Margin

Especifica el espacio entre este control y el margen de otro control

4 MultiColumn

Indica si los valores deben mostrarse horizontalmente por columnas.

5 Sorted

Controla si la lista esta ordenada.

Metodos

```
ListBox mylist = new ListBox();  
mylist.Location = new Point(287, 109);  
mylist.Size = new Size(120, 95);  
mylist.ForeColor = Color.Purple;  
mylist.Items.Add(123);  
mylist.Items.Add(456);  
mylist.Items.Add(789);
```

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.

ListView

Propiedades

1 Activation

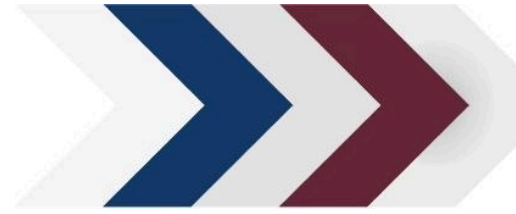
Indica el tipo de acción que debe realizar el usuario para activar un elemento y los comentarios proporcionados.

2 Columns

Columnas mostrada en la vista de detalles.

3 Groups





Grupos de ListView.

4 MultiSelect

Permite seleccionar varios elementos.

5 OwnerDraw

Controla si el sistema o el usuario pinta los elementos o sub-elementos.

Metodos

OnLostFocus(EventArgs)

OnMarginChanged(EventArgs)

OnMouseCaptureChanged(EventArgs)

OnMouseClicked(MouseEventArgs)

OnMouseDoubleClick(MouseEventArgs)

OnMouseDown(MouseEventArgs)

OnMouseEnter(EventArgs)

OnMouseHover(EventArgs)

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.

MaskedTextBox

Propiedades

1 AsciiOnly

Indica si los caracteres ASCII se aceptan como entrada valida.

2 Culture

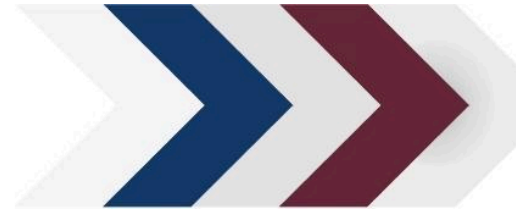
Referencia cultural que determina el valor de los separadores y los marcadores de posición de idioma de la mascara localizable.

2 Mask Establece

la cadena que controla la entrada permitida para este control.

3 PasswordChar





Indica el carácter que se va a mostrar para la entrada de contraseña.

4 TextMaskFormat

Indica si la cadena devuelta de la propiedad Text incluye literales o caracteres de petición de datos.

Metodos

OnLostFocus(EventArgs)

OnMarginChanged(EventArgs)

OnMouseCaptureChanged(EventArgs)

OnMouseClicked(MouseEventArgs)

OnMouseDoubleClick(MouseEventArgs)

OnMouseDown(MouseEventArgs)

OnMouseEnter(EventArgs)

OnMouseHover(EventArgs)

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.

MonthCalendar

Propiedades

1 AnnuallyBoldedDates

Indica que fechas anuales se deben poner en negrita.

2 FirstDayOfWeek

Primer día de la semana.

3 MaxDate

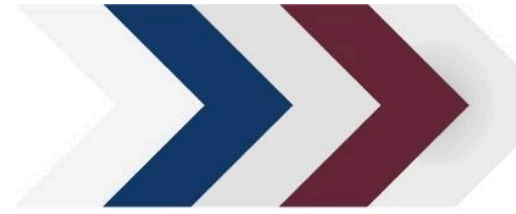
Fecha máxima que se puede seleccionar para un control de calendario mensual.

4 MiniDate

Fecha mínima que se puede seleccionar para un control de calendario mensual.

5 TodayDate





Día actual.

Metodos

OnLostFocus(EventArgs)

OnMarginChanged(EventArgs)

OnMouseCaptureChanged(EventArgs)

OnMouseClicked(MouseEventArgs)

OnMouseDoubleClick(MouseEventArgs)

OnMouseDown(MouseEventArgs)

OnMouseEnter(EventArgs)

OnMouseHover(EventArgs)

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.

NotifyIcon

Propiedades

1 BallonTipIcon

Icono que se va a asociar con la información sobre herramientas de globo.

2 BallonTipText

Texto que se va a asociar con la información sobre herramientas de globo

3 BallonTipTile

Título de la información sobre herramientas de globo.

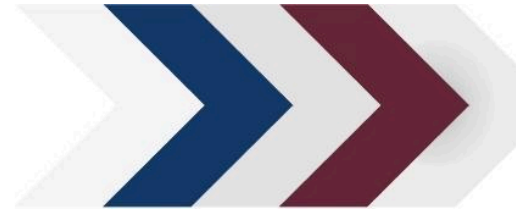
4 ContextMenuStrip

El menú contextual que se mostrara cuando el usuario haga clic con el botón secundario en el icono.

5 GenerateMember

Indica si se genera una variable miembro para este componente.





Metodos

OnLostFocus(EventArgs)
OnMarginChanged(EventArgs)
OnMouseCaptureChanged(EventArgs)
OnMouseClicked(MouseEventArgs)
OnMouseDoubleClick(MouseEventArgs)
OnMouseDown(MouseEventArgs)
OnMouseEnter(EventArgs)
OnMouseHover(EventArgs)

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

Este evento ocurre cuando el puntero del ratón descansa sobre el control.

NumericUpDown

Propiedades

1 DecimalPlaces

Indica el numero de posiciones decimal que se muestra.

2 Dock Define

los bordes del control enlazados al contenedor.

3 Hexadecimal

Indica si el control numérico de fichas debe mostrar el valor como hexadecimal.

4 Increment

Indica la cantidad que se va a aumentar o disminuir cada vez que hace clic en el botón.

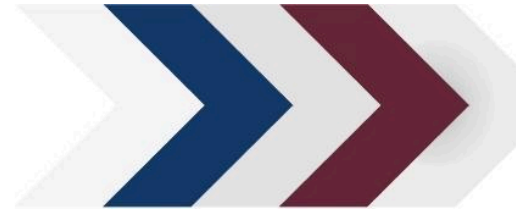
5 Modifiers

Indica el nivel de visibilidad del objeto.

Metodos

OnLostFocus(EventArgs)
OnMarginChanged(EventArgs)





OnMouseCaptureChanged(EventArgs)
OnMouseClicked(MouseEventArgs)
OnMouseDoubleClick(MouseEventArgs)
OnMouseDown(MouseEventArgs)
OnMouseEnter(EventArgs)
OnMouseHover(EventArgs)

Eventos

Leave

Esto ocurre cuando el enfoque de entrada sale del control.

MouseClicked

Esto ocurre cuando el ratón pulsa el control.

MouseDoubleClick

Esto ocurre cuando el ratón hace doble clic en el control.

MouseDown

Este evento ocurre cuando el puntero del ratón está sobre el control y se pulsa un botón del ratón.

MouseEnter

Este evento ocurre cuando el puntero del ratón entra en el control.

MouseHover

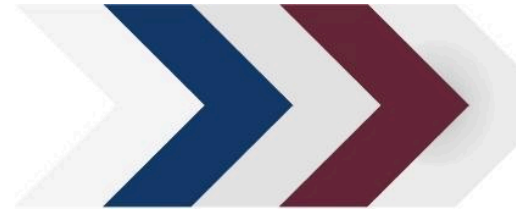
Este evento ocurre cuando el puntero del ratón descansa sobre el control.

16) Un procedimiento (conocido como método void en C# o Sub en VB.NET) es un bloque de código que ejecuta un conjunto de instrucciones para realizar una tarea específica, pero no devuelve ningún valor al bloque que lo invocó. Se declara indicando su nivel de acceso (ej. public, private), seguido de la palabra reservada que indica que no hay retorno (void), el nombre del procedimiento y paréntesis para los parámetros.

Ejemplo en C#:

```
public void MostrarSaludo(string nombre)
{
    MessageBox.Show("Hola " + nombre);
}
```





17) Una función es un bloque de código similar al procedimiento, pero con la característica obligatoria de que debe devolver un valor (un resultado) al código que la invocó. Se declara indicando el nivel de acceso, el tipo de dato que va a retornar (ej. int, string, bool), el nombre de la función y los parámetros. Dentro de su bloque debe existir la instrucción return.

Ejemplo en C#:

```
public int SumarNumeros(int numero1, int numero2)
{
    int resultado = numero1 + numero2;
    return resultado;
}
```

18) La diferencia principal está en el retorno de datos:

- La función calcula un resultado y lo devuelve al código que la llamó para que este pueda almacenarlo en una variable o utilizarlo en otra expresión.
- El procedimiento simplemente ejecuta una acción (como escribir en un archivo, cambiar el color de un botón o mostrar un mensaje en pantalla) y termina su ejecución sin devolver ningún dato al bloque que lo originó.

19) Pasar un parámetro "por valor" significa que el procedimiento recibe una copia exacta del dato original. Si dentro del procedimiento se modifica el valor de ese parámetro, este cambio solo tiene efecto de manera local (dentro del procedimiento). La variable original que se envió desde afuera permanece intacta y conserva su valor original. Es el comportamiento por defecto en lenguajes como C# para los tipos de valor (como enteros, booleanos o decimales).

20) Pasar un parámetro "por referencia" significa que no se envía una copia del dato, sino la dirección de memoria donde se encuentra la variable original. En consecuencia, cualquier modificación que la función le haga a ese parámetro internamente afectará de manera directa e irreversible a la variable original externa. En C#, esto se indica explícitamente utilizando palabras clave como ref u out en los parámetros.





21)

- if / else if / else: Evalúan una condición de tipo booleana y, si es verdadera, ejecutan su bloque de código correspondiente; de lo contrario, evalúan las ramas alternativas o ejecutan el bloque else final.
 - `if (edad >= 18) { Console.WriteLine("Mayor"); }`
- switch: Compara una misma variable o expresión contra múltiples casos o valores constantes posibles.
 - `switch(turno) { case "Mañana": ... break; }`
- Operador Condicional (?): Es una estructura de decisión simplificada en una sola línea que evalúa una condición, devolviendo un valor si es verdadera y otro si es falsa.
 - `int mayor = (a > b) ? a : b;`

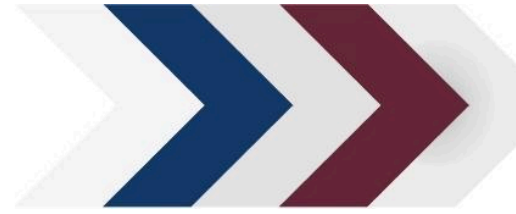
Diferencias: El if / else if / else es la estructura más general y flexible, permitiendo evaluar expresiones complejas combinando operadores lógicos y comparaciones de rangos. El switch se prefiere por legibilidad y prolijidad cuando se compara una única variable contra múltiples valores discretos y constantes. El operador condicional (?) no sirve para ejecutar bloques de código complejos, sino para resolver elecciones y asignaciones directas de valor dentro de una expresión.

22)

- for: Se usan cuando la cantidad de iteraciones es conocida de antemano.
 - `for (int i = 0; i < 5; i++) { ... }`
- while: Evalúa una condición lógica antes de cada iteración y repite el código interno mientras dicha condición siga siendo verdadera.
 - `while (x < 10) { x++; }`
- do-while: Evalúa la condición lógica después de ejecutar el cuerpo del bucle, garantizando que el código interno se ejecute al menos una vez.
 - `do { x++; } while (x < 10);`
- foreach: Diseñado específicamente para recorrer de forma directa cada uno de los elementos de una colección o de un array.
 - `foreach (var nota in notas) { ... }`

Diferencias: El ciclo for integra de forma compacta el índice de control, la condición de parada y la expresión de incremento, lo cual es ideal para recorrer rangos numéricos definidos. El ciclo while depende puramente de una condición y puede no ejecutarse nunca si la condición de entrada es falsa. El ciclo do-while funciona igual que el while pero con la garantía de realizar al menos una iteración. El ciclo foreach facilita la iteración en colecciones al ocultar por completo el manejo de índices o contadores manuales.





23)

Se crea usando la palabra struct para agrupar campos o variables relacionadas en un nuevo tipo personalizado. En C#, una estructura es un tipo por valor, lo que significa que guarda su información directamente dentro de su asignación en la memoria Stack (Pila) y, cuando se asigna a otra variable, se realiza una copia idéntica del dato.

Ejemplo:

```
struct Punto
```

```
{
```

```
    public int X;
```

```
    public int Y;
```

```
}
```

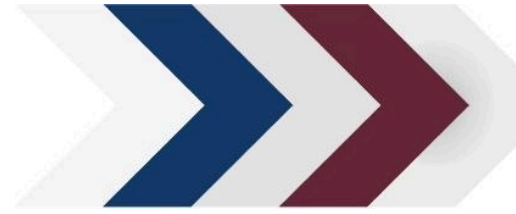
24)

Una propiedad es un miembro que expone el estado de un objeto o de un control visual (por ejemplo, propiedades comunes de controles como su color, tamaño, visibilidad o texto de interfaz). Estas actúan como una capa de control que permite implementar lógica de lectura y escritura personalizada mediante bloques de acceso get y set.

25) Un método es una función o procedimiento que define el comportamiento o las acciones que puede realizar un objeto o control (como Focus() para enfocar un control o Clear() para vaciar un cuadro de texto).

En C# se dividen principalmente en dos de acuerdo con su tipo de retorno:

- Procedimientos: Se declaran con el tipo de retorno void, ejecutan una serie de instrucciones y no devuelven ningún valor.
- Funciones: Se declaran especificando de forma explícita el tipo de dato que van a retornar, requieren obligatoriamente la instrucción return y devuelven un valor que puede ser utilizado directamente en expresiones del programa.



26) Es una notificación que un objeto envía cuando ocurre algo sobre él (un clic, un cambio de texto, el paso de un intervalo de tiempo en un Timer, etc.). En C# un evento se declara con la palabra clave event y se basa en delegados: el control "dispara" el evento y cualquier método que esté suscrito a él se ejecuta en respuesta.

Ejemplo:

```
private void btnAceptar_Click(object sender, EventArgs e)
{
    MessageBox.Show("Botón presionado");
}
```

27) Para representar estados que solo pueden tomar dos valores posibles: true o false. Se usa típicamente como bandera de control, resultado de una validación o condición dentro de estructuras if/while (por ejemplo, bool formularioValido, bool estaHabilitado).

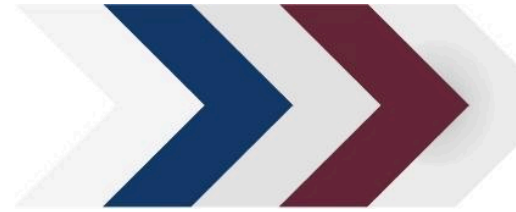
28) Se utiliza para convertir una cadena de texto (string) a su valor numérico equivalente del tipo correspondiente (en este caso decimal), permitiendo luego operar matemáticamente con un dato que originalmente se ingresó o almacenó como texto (por ejemplo, el contenido de un TextBox).

29) decimal[] vector = new decimal[10];

30) Boxing es la conversión implícita de un tipo por valor (value type, como int o decimal) a un tipo por referencia (object).

Unboxing es el proceso inverso: convertir explícitamente ese object de vuelta a su tipo de valor original, lo cual requiere un cast explícito.

Ejemplo:



```
//Escribir un programa de consola en C# que declare una variable int,  
//la convierte a object (boxing) y luego la reconvierta a int (unboxing).  
//Mostrar por consola el tipo (GetType()) en cada caso  
  
int nro = 1;  
  
Console.WriteLine("El valor original es: " + nro);  
Console.WriteLine("El tipo original es: " + nro.GetType());  
  
object box = nro;  
  
Console.WriteLine("\n\nBOXING");  
Console.WriteLine("\nEl valor nuevo es: " + box);  
Console.WriteLine("\nEl tipo original es: " + box.GetType());  
  
int nroNuevo = (int)box;  
  
Console.WriteLine("\n\nUNBOXING");  
Console.WriteLine("\nEl valor nuevo es: " + nroNuevo);  
Console.WriteLine("\nEl tipo original es: " + nroNuevo.GetType());
```

